

1. Écrire à l'écran

Une cartouche qui affiche un mot, et rien d'autre.

1. Ce qu'il faut comprendre

Le plus petit programme qui fait quelque chose. Comme tout programme C++, il commence à `int main()` — c'est là que la console arrive, et nulle part ailleurs.

`texte(colonne, ligne, "...")` écrit à partir d'une case : l'écran en fait **20 de large sur 18 de haut**, et la case (0, 0) est en haut à gauche.

La boucle `while (true)` n'est pas décorative. Un programme qui se termine laisse le processeur partir n'importe où ; ici, il tourne en attendant l'image suivante, soixante fois par seconde, et l'écran reste affiché.

2. Le programme, morceau par morceau

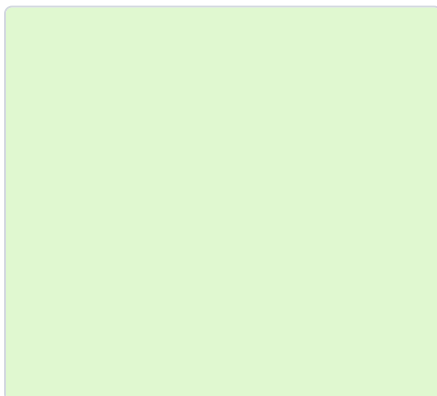
Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1308 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(5, 6, "BONJOUR");  
    texte(3, 9, "GAME BOY");  
  
    while (true) {  
        image();  
    }  
  
    return 0;  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« BONJOUR » à la colonne 5, ligne 6.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ « BONJOUR » est écrit

✓ « GAME BOY » aussi

✓ l'écran est allumé

Ensuite : **2. Effacer ce qu'on a écrit** — niveau 1 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++